

Diagnostico Tecnico

Diagnóstico Técnico do Legado

Contexto

O código original analisado foi a variante Python do repositório `emilybache/GildedRose-Refactoring-Kata`, especialmente o arquivo `python/gilded_rose.py`.

Principais problemas encontrados

1. Excesso de condicionais aninhadas

O método `update_quality()` do legado concentra toda a regra de negócio em uma estrutura extensa de `if/else`, como pode ser visto em `python/gilded_rose.py:8-36`.

Impactos:

- leitura difícil;
- alto custo para localizar regras de cada item;
- maior risco de regressão ao adicionar novo comportamento.

2. Responsabilidades misturadas

O mesmo método decide o tipo do item, altera `quality`, altera `sell_in`, trata expiração e impõe exceções para itens lendários.

Impactos:

- baixa coesão;
- pouca reutilização;
- manutenção mais arriscada.

3. Forte acoplamento a strings literais

As categorias dos itens são reconhecidas diretamente por comparações textuais repetidas, como `Aged Brie`, `Backstage passes...` e `Sulfuras...`.

Impactos:

- duplicação de conhecimento;
- dificuldade para adicionar novas categorias como `Conjured`;
- maior chance de erro por digitação.

4. Duplicação de regras

A degradação de qualidade aparece em mais de um ponto do método, inclusive em blocos diferentes para antes e depois do vencimento.

Impactos:

- comportamento espalhado;
- dificuldade de evolução;
- necessidade de validar vários trechos para uma mudança simples.

5. Baixa testabilidade inicial

O projeto original traz:

- um teste placeholder em `python/tests/test_gilded_rose.py`;
- um teste por approval em `python/tests/test_gilded_rose_approvals.py`, dependente de configuração adicional.

Na prática, a suíte inicial não oferece proteção suficiente para refatoração segura.

6. Nova regra ausente no legado

O fixture original já lista `Conjured Mana Cake`, mas o código legado trata esse item como se fosse um item comum. Isso indica um gap explícito entre requisito e implementação.

Conclusão do diagnóstico

O sistema original funciona para boa parte das regras básicas, mas sua estrutura dificulta manutenção, testes e extensão. A modernização precisou priorizar:

- isolamento das regras por tipo de item;
- centralização de invariantes de domínio;
- criação de testes automatizados antes da refatoração estrutural;
- implementação específica dos itens `Conjured`.